

OmniDrive — Plataforma P2P de renta de vehículos

| Versión MVP — Ecuador · USD · Desarrollado en 7 días

Stack

Capa	Tecnología
Backend	Node.js 20 + Express + TypeScript + Prisma
Base de datos	PostgreSQL (Supabase)
Frontend	React 19 + Vite 6 + Tailwind 4 + TypeScript
Auth	JWT manual (bcryptjs)
Storage	Supabase Storage
Pagos	Stripe Connect + Wallet interna
Push	Web Push (VAPID) + Service Worker
Deploy	Railway (backend) + Netlify (web)

Inicio rápido

Requisitos

- Node.js 20+
- PostgreSQL (o cuenta Supabase)
- Cuenta Stripe
- Cuenta Supabase

1. Clonar y configurar

```
bash
git clone https://github.com/tu-usuario/omnidrive.git
cd omnidrive
```

2. Backend

```
bash

cd backend
cp .env.example .env
# → Editar .env con tus credenciales

npm install
npx prisma db push          # Crea las tablas
npm run db:seed             # Carga datos de demo
npm run dev                 # Inicia en http://localhost:3000
```

3. Frontend

```
bash

cd web
cp .env.example .env
# → Editar .env con VITE_API_URL y VITE_VAPID_PUBLIC_KEY

npm install
npm run dev                 # Inicia en http://localhost:5173
```

4. Generar claves VAPID (una sola vez)

```
bash

cd backend
npm run vapid:keys
# → Copia Public Key a backend/.env y web/.env
# → Copia Private Key solo a backend/.env
```

Cuentas de demo (después del seed)

Rol	Email	Contraseña
Admin	admin@omnidrive.ec	Admin1234!
Owner	carlos@demo.ec	Owner1234!

Rol	Email	Contraseña
Tenant (Premium)	<u>sofia@demo.ec</u>	Tenant1234!

Flujo completo

Registro → Verificación de identidad (admin)

- Publicar vehículo
- Buscar por fechas/ubicación/categoría
- Reservar (con/sin seguro, con/sin chofer)
- Owner confirma (acepta waiver si no hay seguro)
- Iniciar viaje (PIN de 4 dígitos)
- GPS tracking cada 30s (owner ve ruta en tiempo real)
- Finalizar → pago liberado al owner (-15% comisión)
- Review mutua → driver score actualizado

Endpoints del backend

Módulo	Base	Endpoints
Auth	<code>/api/auth</code>	register, login, me, verify-identity
Users	<code>/api/users</code>	perfil público, vehículos, reviews
Vehicles	<code>/api/vehicles</code>	CRUD, fotos, disponibilidad
Bookings	<code>/api/bookings</code>	CRUD, confirm, start, end, fotos, disputa
Payments	<code>/api/payments</code>	wallet, deposit, withdraw, hold/release/refund
Subscriptions	<code>/api/subscriptions</code>	planes, suscribir, cancelar
Tracking	<code>/api/tracking</code>	report GPS, get ruta
Reviews	<code>/api/reviews</code>	crear, listar por usuario
Admin	<code>/api/admin</code>	users, vehicles, bookings, metrics, disputas
Push	<code>/api/push</code>	subscribe, unsubscribe

Módulo	Base	Endpoints
Notifications	<code>/api/notifications</code>	list, mark-read, mark-all-read
Stripe	<code>/api/stripe/webhook</code>	payment_intent, account.updated

Deploy en producción

Backend → Railway

1. Crear proyecto en railway.app
2. New Service → GitHub → seleccionar `/backend`
3. Add PostgreSQL plugin (Railway genera `DATABASE_URL` automáticamente)
4. Variables de entorno → copiar de `backend/.env.example`
5. Deploy automático en cada push a `main`

```
bash

# Migración en producción (ejecutar desde Railway CLI o en build command)
npx prisma migrate deploy
npm run db:seed
```

Web → Netlify

1. Crear site en netlify.com → Import from Git
2. Base directory: `web`
3. Build command: `npm run build`
4. Publish directory: `dist`
5. Variables de entorno → copiar de `web/.env.example`
6. Deploy automático en cada push a `main`

Stripe

1. dashboard.stripe.com → API Keys
2. Webhooks → Add endpoint → `https://[railway-url]/api/stripe/webhook`
3. Eventos: `payment_intent.succeeded`, `payment_intent.payment_failed`, `account.updated`

Supabase Storage

1. Crear proyecto en supabase.com
 2. Storage → New bucket → nombre: `omnidrive` → Public: ON
 3. Settings → API → copiar `URL` y `service_role` key
-

Tests E2E

```
bash

cd backend

# Con servidor local corriendo:
npm run test:e2e

# Contra producción:
API_URL=https://omnidrive-api.railway.app/api npm run test:e2e
```

El test prueba el flujo completo de 18 pasos:

registro → verificación → vehículo → reserva → confirm → start → GPS → end → review → admin

Variables de entorno

backend/.env

```
env
```

```
DATABASE_URL=postgresql://...
JWT_SECRET=...
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_SERVICE_KEY=...
SUPABASE_BUCKET=omnidrive
STRIPE_SECRET_KEY=sk_...
STRIPE_WEBHOOK_SECRET=whsec_...
VAPID_PUBLIC_KEY=B...
VAPID_PRIVATE_KEY=...
VAPID_EMAIL=admin@omnidrive.ec
FRONTEND_URL=https://omnidrive.netlify.app
PORT=3000
NODE_ENV=production
```

web/.env

```
env

VITE_API_URL=https://omnidrive-api.railway.app/api
VITE_VAPID_PUBLIC_KEY=B...
VITE_MAPBOX_TOKEN=pk.eyJ1...
VITE_STRIPE_PK=pk_...
```

Estructura del proyecto

```
omnidrive/
├── backend/
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── seed.ts
│   ├── src/
│   │   ├── index.ts
│   │   ├── routes/          (11 módulos)
│   │   ├── middleware/     (auth, validate)
│   │   ├── services/       (stripe, wallet, push, tracking)
│   │   ├── lib/            (prisma, storage)
│   │   └── tests/          (e2e.test.ts)
│   ├── Dockerfile
│   ├── railway.toml
│   └── package.json
```

```
├─ web/
|   ├─ public/
|   |   ├─ sw.js          (Service Worker)
|   |   ├─ offline.html
|   |   └─ manifest.webmanifest
|   └─ src/
|       ├─ App.tsx
|       ├─ pages/        (10 páginas)
|       └─ components/   (Layout, Navbar, VehicleCard, NotificationBell,
PushBanner)
|           ├─ hooks/    (usePush, useNotifications)
|           └─ lib/      (api, store, push)
├─ netlify.toml
├─ package.json
├─ scripts/
|   └─ deploy.sh
└─ README.md
```

Consideraciones Ecuador

- Precios en USD (moneda oficial)
- Documentos: cédula (10 dígitos) o pasaporte
- Edad mínima: 21 años (validar en frontend al registrar)
- SOAT: campo `insurance` en vehículo, verificación manual por admin
- Licencia tipo B para autos, tipo A para motos

OmniDrive MVP — Construido en 7 días 🚗 ⚡